

Flask를 이용한 YOLOv5 스트리밍 서버 작성

작성자: 서예현

웹서비스 구현에 앞서

사용한 도구

Flask를 이용한 웹페이지 작성

요구사항

사용자 기능 구현

이미지 관련 기능 구현

프로젝트를 마치며

웹서비스 구현에 앞서

DW 아카데미에서 수학을 시작하고 약 2달이 됐을 때, Flask를 이용한 웹서비스를 배우게 되었다. 지금껏 배워온 것을 이용하여 눈에 보이는 결과물이 만들어진다는 것은 즐거운 경험이었다. 해당 경험을 문서적으로 남기고자 본 리포트의 작성을 시작하였으나, 작성 과정 중 국립과천과학관을 방문하는 일이 생겼다. 그곳에서 우연히 현재 학습 중인 딥러닝 기반의 객체 감지 기술의 전시를 보게 되었다. 강의실에서 배운 기술과, 활자적으로 알고 있던 목표 분야를 실제 기술로서 접하는 것은 큰 동기부여가 되었다. 이에 복습과 기록의 목적으로 시작하였던 리포트를 직접 코드를 재작성하여 완전히 이해해보고자 한다.

사용한 도구

Language: Python 3.10

Framework: Flask

Library: YOLOv5

DataBase: SQLAlchemy, Migrate

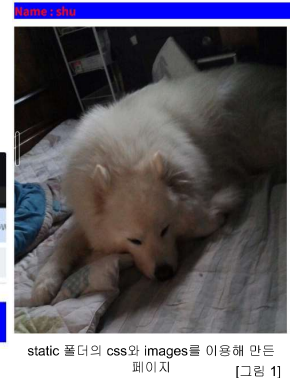
Module: Torch, Torchvision, OpenCV, numpy, PIL, os

Flask를 이용한 웹페이지 작성

본격적으로 웹서버를 작성하기에 앞서
우선적으로 Flask의 기능에 대해
알아보았다. Flask를 이용할 때는 static
폴더를 이용하여 CSS등의 디자인을 입히는
것이 가능하다.

또한 static을 이용할 경우, 페이지에
이미지를 출력하는 것 또한 가능했다. 다만

static을 이용하여 이미지를 출력할 경우, static에 이미지를 저장해둬야만 했다.



[그림 2] Flask를 이용한 객체 감지 결과

최종적으로 목표하는 것은
[그림 2]와 같은 YOLO를
이용한 객체 감지를 실시하는
웹 사이트이다.
창문 너머 베란다에 갇혀있는
고양이와 그 고양이를 약
올리는 사람의 손, 책상 위의
물병이나 모니터(TV), 마우스
등을 잘 인식하고 있는 것을
확인할 수 있다.

아카데미에서 작성한 코드가 제대로 작동되는 것을 확인하였으므로, 이번에는 아카데미에서 수행하였던 코드 작성을 직접 해보고자 한다.

요구사항

코드 작성에 앞서, 구현하고자 하는 기능은 다음과 같다.

초기 화면: 신규 등록, 로그인 버튼, 이미지 업로드 버튼, 영상 감지 버튼, 업로드 된 이미지 출력.

신규 등록: 사용자 이름, 이메일, 비밀번호 작성란, 회원 가입 버튼.

로그인 화면: 사용자 이메일, 비밀번호 작성란, 로그인 버튼.

이미지 관리 및 객체 감지: 업로드 된 이미지의 객체 감지, 삭제 버튼.

신규 등록, 로그인 버튼 외에는 모두 로그인 한 사용자만이 이용 가능하도록 만들고자 한다.

사용자 기능 구현

의도와는 달리, 시작하자마자 [그림 4]와 같은 오류 메시지를 직면하였다.

원인을 파악한 결과, 해결 방법은 간단했다.

blueprint의 설정 상으로는 base.html과 index.html 파일을 templates/auth 폴더에 위치하여야 했는데, 그냥 templates 파일에 넣어두었던 것이다.

해당 파일들의 경로를 수정해주었더니, [그림 6]에서 보이듯 페이지가 로딩 되는 것을 확인할 수 있었다. 이는 아카데미에서도 발생했던 실수였는데, 이번에도 똑같은 실수를 반복하였다. 경로의 설정을 철저히 확인해야 함을 다시금 확인할 수 있었다.

페이지를 띄우는 데에 성공했으니, 본격적으로 기능들을 구현해보고자 한다. 초기 화면과 사용자 관련 기능들은 auth 폴더를 이용할 것이다.

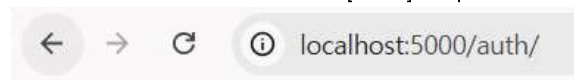
Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

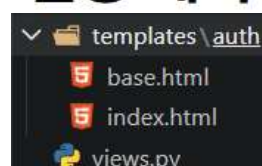
[그림 4] 로그인 페이지 작성 시작 즉시 마주한 에러 메시지

```
auth=Blueprint("auth",
                __name__,
                template_folder="templates",
                static_folder="static"
            )
```

[그림 5] Blueprint의 경로



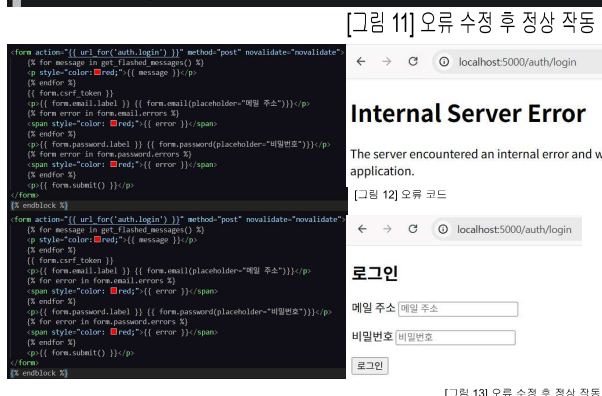
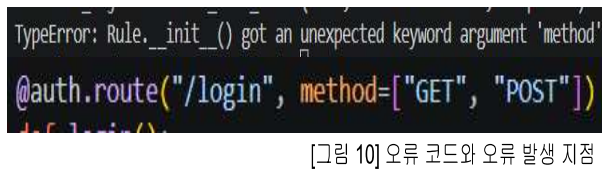
인증 페이지 표시



[그림 6] 해결한 이후 정상 작동

사용자가 곧 관리자와 같은 역할을 하므로, crud 폴더는 과감하게 삭제하기로 하였다.

회원가입 기능을 만들기 위해서 Flask-Login 기능을 설치하였다.



이번 시도에는 함수에 괄호를 입력하는 것을 잊은 탓에 오류가 발생하였다. 심지어 이 문제를 찾지 못한 탓에 장장 1시간을 코드를 눈이 빠져라 들여다보았다. 결국 아카데미에서 작성하였던 코드와 비교하며, 겨우 해당 오류를 발견하고 수정할 수 있었다. 오류를 수정하자마자 페이지는 정상적으로 로딩 되었으며, 성공적으로 사용자 신규 등록의 화면을 만들 수 있었다. 설정해둔 오류 메시지도 출력되는 것을 확인하였으니, 이번에는 로그인 화면을 만들고자 한다.

로그인 화면 구축 중에도 오타로 인한 오류가 발생하였다. 다행히 이번 오류는 오타를 위주로 보았고, 코드의 길이가 그리 길지 않은 덕분에 빠르게 확인하는 것이 가능했다.

그러나 views의 오타를 수정했음에도 로그인 화면은 로딩 되지 않았다. 다시금 아카데미 코드와 비교하여 확인한 결과, login.html의 {% for error ~ %} 구문의 for를 모두 form으로 적었음을 확인할 수 있었다. 해당 부분을 for로 수정한 결과, 로그인 화면이 정상적으로 작동함을 확인 가능하였다.

사용자 신규 등록 화면과 로그인 화면을 구축하는 과정 중 발생한 오류들은 모두 오타자로 인한 문제였다. 아카데미에서도 대부분의 오류는 오타라는 이야기를 들었으나,

그 빈도에 대해 큰 감흥이 없었는데, 이렇게 스스로 코드를 작성해보니 얼마나 많은 오타자가 개발자들의 적이 되는지 알 수 있을 것만 같았다. 작성을 마친 후 퇴고를 거치는 작가들처럼, 개발자로서의 길을 가기 위해서는 코드를 작성한 이후 오타자를 찾고, 각 언어의 문법을 확고히 숙지하는 과정이 필수적이라는 것을 뼈저리게 깨닫는 중이다.

로그인 화면의 작동을 확인한 이후, 로그아웃 기능을 만들지 않았음을 깨달았다. 요구사항 정의를 제대로 해야하는 이유를 알게 되었다. 로그아웃 버튼을 만들며, 회원 등록 후 로그인 화면이 라우팅 되도록 코드를 일부 수정하기도 하였다.

그러나 이번에도 오류가 발생했다. 이번에는 {% endif %} 구문을 적지 않아 발생한 오류였다. 오류를 수정하고 테스트를 진행하기 위해 사용자 등록을 진행하였다.

[그림 14] 오류 코드와 발생 지점

[그림 15] 오류 해결 후 정상 작동

[그림 16] 데이터 베이스 오류 발생

회원 등록을 하는 순간, 처음 보는 오류가 발생하였다. 그러나 SQLAlchemy가 데이터베이스 모듈임을 알고 있었으므로, 직감적으로 데이터베이스의 문제일 것임을 알 수 있었다. flask db upgrade 명령어를 통하여 데이터베이스를 갱신하였더니, 회원가입부터 로그인까지 막히지 않고 진행되었다. 그림 17에 보이는 로그아웃 버튼을 클릭하니, 다시 로그인 화면으로 돌아가는 것 또한 확인할 수 있었다.

[그림 18] 오류 코드와 오류 구문

test [로그아웃](#)

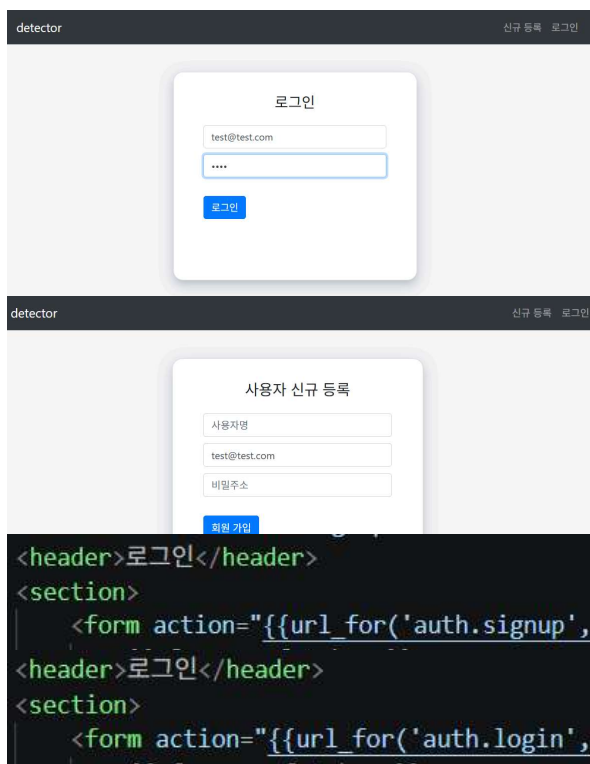
인증 페이지 표시

[그림 17] 로그아웃 버튼이 정상적으로 표시된다.

이제 초기 화면을 구성해보고자 한다. 초기 화면에서는 헤더를 이용하여 신규 등록 및 로그인 기능이 가능하도록 만들 계획이다.

우선 detector 폴더를 만들어, 초기 화면으로 사용할 라우터를 작성하였다. 새로운 초기 화면은 빠르게 만들었으나, 오류가 발생하였다. 이는 app에서 인스턴스명을 잘못 작성한 것이 원인이었다. 이에 해당 부분을 detector 폴더의 views 파일에서 사용한 인스턴스명인 dt로 수정하니 정상적으로 페이지가 로딩되는 것을 확인할 수 있었다.

이어 auth 폴더의 views 파일을 수정하여, 로그인 시 라우팅 되는 화면을 detector의 index로 변경하였다. 테스트용으로 만든 계정으로 로그인을 시도한 결과, 이번에는 오류 없이 초기 화면으로 라우팅 되는데에 성공하였다.



[그림 21] 오류 발생 구문



[그림 22] 오류 수정 후 정상 작동 확인

초기 화면에 헤더를 등록하여 신규 등록, 로그인 버튼을 띄우는 것은 성공하였으나, 어째서인지 로그인 시 사용자 신규 등록 창으로 이동 되는 오류가 발생하였다. 수정한 코드가 많은 탓에 원인을 찾아내는데에 시간이 걸렸으나, 결국 해결하는데에 성공할 수 있었다. views 파일에서 라우팅 경로 설정을 실수했거나, app 파일의 설정상 오류일거라 생각했던 내 예상과 달리, 원인은 로그인 화면의 html이었다. 회원 가입 화면의 코드를 그대로 가져오며 auth.signup 부분을 수정하지 않았고, 이로 인해 오류가 발생하게 된 것이었다. 이를 수정함으로써 오류는 발생하지 않게 되었고, 의도했던 대로 회원 가입 후 로그인을 진행할 시 초기 화면으로 이동되는 것을 확인할 수 있었다.

웹 사이트의 구현을 완료하였으므로, 이제는 이미지 관련 기능들을 구현하고자 한다. 이미지

관련 기능으로는 이미지의 업로드와 삭제, YOLO를 이용하여 업로드 한 이미지의 객체 감지, 영상을 이용한 실시간 객체 감지 등을 구현하려 한다. 이에 더하여 각 기능들로 연결되는 버튼을 함께 만들고자 한다.

이미지 관련 기능 구현

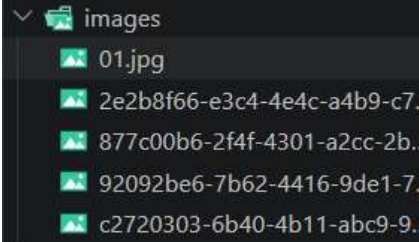
이미지 업로드 화면을 만들던 중 발생한 오류는 그림 23에서 보이는 것과 같다. 모두 import 관련 오류로, 잘못된 모듈을 가져오거나, 아예 import를 누락한 경우가 잦았다. 이를 해결하기 위해 모듈과 객체를 제대로 import하니 정상적으로 이미지가 업로드 되었다.

그러나 의도와는 달리 초기 화면에서 업로드 된 이미지가 출력되지 않았다. 이를 수정하기 위해 아카데미의 코드를 확인하였으나, 완성형 코드인 아카데미 코드에서는 해당 부분을 이후의 기능을 위해 수정한 상태였고, 원하는 내용을 찾을 수 없었다. 결국 교재와 자료를 찾아 index 라우트에 문제가 있음을 알아내고, 해당 라우트를 수정하여 문제를 해결할 수 있었다.

```
from flask_login import current_app, login_required
ImportError: cannot import name 'current_app' from 'flask_login'
NameError: name 'UserImage' is not defined
NameError: name 'db' is not defined
```

[그림 23] 오류 코드

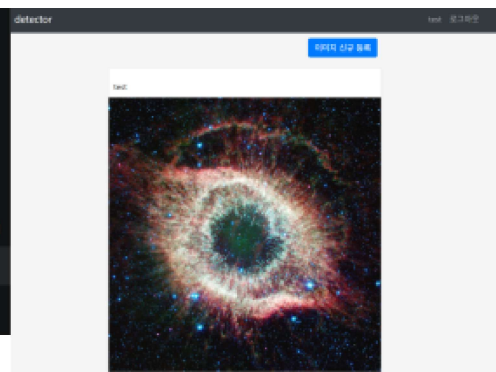
```
from flask_login import current_user, login_required
from apps.detector.models import UserImage
from apps.app import db
from flask import render_template, request, url_for
import os
```



[그림 24] 오류 해결 후 이미지 업로드 확인

```
@dt.route("/")
def index():
    user_images=(
        db.session.query(User, UserImage)
        .join(UserImage)
        .filter(User.id==UserImage.user_id)
        .all()
    )
```

[그림 25] 이미지 일람 수정 코드(좌)와 업로드 한 이미지가 출력되는 초기화면(우)

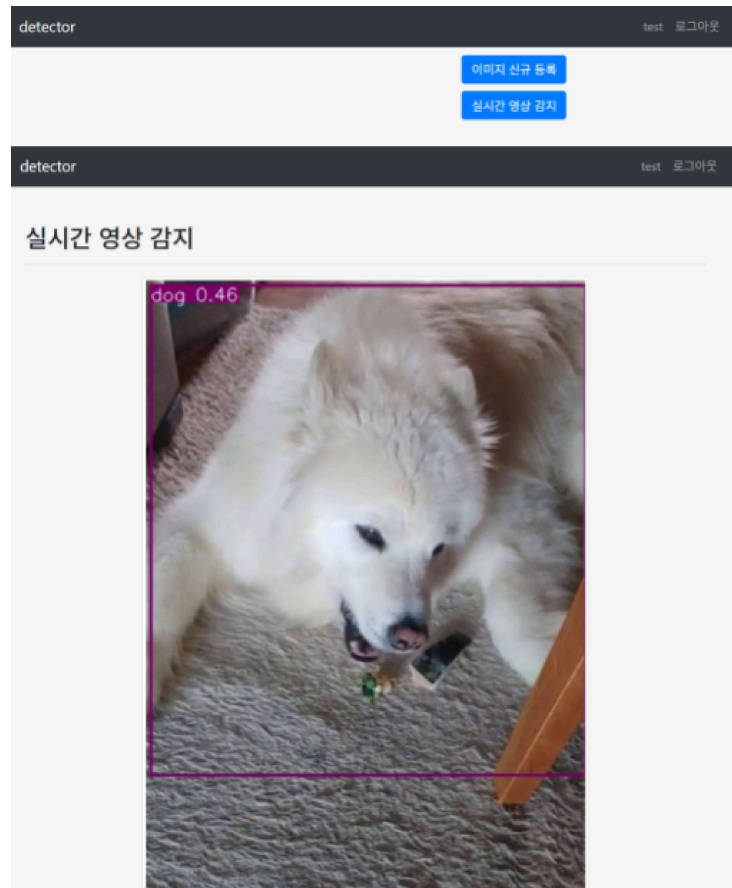


객체 감지 버튼을 작성하기에 앞서, 업로드한 이미지를 삭제하기 위한 삭제 버튼을 추가하였다. 해당 버튼을 누를 경우 등록 되어 있던 이미지가 삭제 되는 것 또한 확인 가능하였다.

이후, 정지된 이미지 내부의 객체를 탐지하는 객체 감지 버튼도 추가하려 하였으나, YOLO의 강점인 실시간 객체 탐지를 살리기 위하여 해당 기능은 생략하기로 하였다. 웹 서비스를 제작하는 이유는 실시간 영상을 통한 객체 탐지였으므로, 탐지된 이미지를 저장하는 것이 아닌 저장된 이미지의 객체를 탐지하는 것은 의미가 없을 것이라고 생각하였다.

마지막으로 구현한 기능은 실시간 영상 감지 버튼을 통한 스트리밍 화면 라우팅 이었다. 영상 파일의 경로를 수정하지 않는 실수로 인해 영상 재생이 불가능한 상황이 발생하였으나, 웹 서비스를 구현하는 과정에서 사용하였던 기능들을 활용한 기능인 만큼, 생각만큼 애를 먹지는 않았다.

이로서 계획 하였던 요구사항 중 하나를 제외한 모든 기능을 구현하는데 성공하였다.



[그림 27] 목표했던 기능의 구현 확인

프로젝트를 마치며

Flask를 이용하여 코드를 재작성하며, 얼마나 많은 오류가 얼마나 다양한 원인으로 일어날 수 있는지 알 수 있었다. 코드를 작성하며 오타자에 신경을 쓰면 다른 요소를 놓쳐 오류가 발생하는 일이 빈번하였다. 그러나 해당 문제를 제외한 대부분의 오류는 아직 언어의 문법에 익숙지 않아 발생한 것으로 보였다. 이러한 문제를 예방하기 위해서 지금보다 더욱 파이썬을 비롯한 프로그래밍 언어에 적응하여야 함을 깨달을 수 있었다.

오류로 인한 디버깅의 여정과는 별개로, 스스로 코딩하는 것은 즐거운 경험이었다. 해당 경험을 살려 관심 있는 분야에 한 발 더 나아갈 수 있는 힘을 얻었다는 점이 가장 기뻐다. 그러나 코딩을 진행하는 과정에서 요구 사항을 몇 차례 수정해야만 했고, 이를 통하여 사전에 어떠한 기능을 구현할 것인지 정리하는 요구사항 정의 과정의 중요성을 깨닫는 기회가 되었다.

이번 경험을 토대로 삼아 다음 웹서비스 구현때는 의도한 기능을 모두 구현할 수 있도록 노력하려 한다.